Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Queries and Filters

Learn more at Stellar Cyber Academy.

The following link takes you to a course on the Stellar Cyber Academy technical training portal where you can learn more about this topic by watching the suggested lessons.

J04 Open XDR Investigation Tools (04h:47m)

Create and Save Queries with the Query Tool (28m:00s)

Learn to create and save custom queries using the Query Tool, then watch a demonstration applying these techniques across the Stellar Cyber Platform. Explore how to build query strings, select relevant fields, and save configurations for efficient recall, enabling consistent search results across dashboards, ATH playbooks, and Case Management without recreating searches from scratch.

P03 Using Query Builder and Alert Filters (45m:00s)

PART 1: Introduction to Query Builder and Alert Filters (08m:15s)

Learn how to use the query builder to create custom queries that refine data searches across logs, alerts, and events in the Stellar Cyber Platform. See how logical operators, field types, and nested conditions help pinpoint relevant security data. Understand how alert filters suppress unnecessary alerts, improving your focus on critical events.

PART 2: Navigating Query Builder in the UI (07m:15s)

Discover where to access the Query and Filter Manager in the Stellar Cyber Platform UI. Learn how to build correlation queries to identify attack patterns and visualize events. See how to create custom charts, threat-hunting views, and filters directly within different platform areas, such as Investigate and Visualize.

PART 3: Building Effective Queries with Query Builder (11m:15s)

Follow a demonstration of creating complex queries for automated threat hunting and dashboards. Learn how to nest conditions, use lookup groups, and apply operators like "greater than" and "within" to refine results. See examples of data exfiltration detection, SQL dumpfile monitoring, and script anomaly queries.

PART 4: Field Types, Logical Operators, and Lookup Lists (07m:45s)

Understand field types like boolean, date, IP, and string, and how they impact query creation. Learn to use logical operators such as "contains," "is in lookup," and "field exists" for precise filtering. See how lookup groups simplify queries by referencing predefined lists of values, such as known malicious IP addresses or domains.

PART 5: Alert Filters (07m:15s)

Learn how alert filters help suppress false positives while retaining raw data for machine learning. See an example of filtering impossible travel alerts by subnet and event type. Understand the importance of testing queries in the query builder before applying them as alert filters to avoid missing important alerts.

The first time you access a link on the portal during a session, you must log in to access content.

The Query and Filter Manager (**System | Saved Objects | Queries and Filters**) is organized into tabbed sections with each section consisting of a table and a builder for queries or filters:

- **Queries table**

  The table displays previously configured queries and displays columns for each query name, tenants for which the query is available, query conditions, and (not shown by default) description. It also shows when the query was created and updated and by whom. There's a column showing where the query is in use and an Actions column with options to **Edit** and **Delete** it.

  The **In Use** column identifies how many features are using a query before you consider modifying or deleting it. An entry of zero indicates a query is not in use. Hovering your cursor over a usage count causes a pop-up panel to appear with a list of all the features that are using it. Any change you make to a query affects all associated features. If you want to delete a query, you must first remove it from all associated features.

- **Alert Filters table**

  The table displays previously configured alert filters and displays columns for each filter name, tenants for which the filter is available, filter conditions, and its status as active or inactive. It also shows when the filter was created and updated and by whom and any notes about the filter and who wrote them. There's a column showing the hit count, which indicates the number of alerts that have been suppressed by the filter, and an Actions column with options to **Edit** and **Delete** the filter.

- **Case Filters table**

  The table displays previously configured case filters and displays columns for each filter name, the tenant for which the filter is available, filter conditions, and its status as active or inactive. It also shows when the filter was created and updated and by whom. The **Description** column can contain information about the purpose, usage, and other relevant details of a case filter—making it easier for you to recall its intent later and enabling other users to understand its function and context. There's also an **Actions** column with options to **Edit** and **Delete** the filter.

- **Query and filter builder**

The query and filter builder is a robust tool that not only lets you construct complex searches and exclusion filters—as the query builders and filter builders on individual feature pages also do—but it also includes **Run**, **Test** and **Save As** functions. The **Run** option for queries lets you test queries before applying them, ensuring you get expected results before leaving the page. The **Test** option provides the same functionality as the **Run** option but it's for testing alert filters and case filters. The **Save As** function lets you make copies of queries and filters and adjust settings, working on up to ten variations in tabbed dialog boxes simultaneously.

Queries, alert filters, and case filters are constructed using similar components from the Lucene search engine library. They are covered in the main sections below:

Using Queries and the Query Builder

Queries are requests for data from the Stellar Cyber database. You can build and execute a range of queries from simple to complex that you can use immediately and save for use and reuse in the future. Over time, you might build a library of queries to use when gathering information for analysis.

Unlike the Search tool, which looks for a search term everywhere, a query lets you specify a field to search, greatly increasing the speed. For example, instead of searching for **china** and getting every place where that term appears, you can build a query for **srcip_geo.countryName: china** and get a list of only places where the source IP address country is China.

Query builders are located in the Query and Filter Manager in **System | Saved Objects | Queries and Filters** 5.3.0 and on various feature pages throughout the interface. Instructions for creating queries in the Query and Filter Manager and on feature pages are provided below.

- **Best Practices for Writing Efficient Queries**
- **Use the Query Builder on Feature Pages**
  - **Create a New Query in the Query Builder on a Feature Page**
  - **Build a Query from Active Filters**
- **Use the Query Builder in the Query and Filter Manager**
  - **Create a New Query in the Query and Filter Manager**
  - **Edit a Previously Saved Query**
  - **Export and Import Queries**
  - **Delete a Previously Saved Query**
- **Query Builder Parameters**
  - **Query Name**
  - **Condition Type**
  - **Add Condition**
  - **Add String Condition**
  - **Add Inner Group**

- Remove This Group
- Use Aggregations in the Query Builder
- Remove

While basic query and filter creation is available on individual feature pages throughout the UI, the Query and Filter Manager provides a single workspace where all queries, alert filters, and case filters are gathered. The Query and Filter Manager also provides advanced options that let you test queries and work with multiple query and filter variations at once. The convenience of having all the queries and filters together plus its robust capabilities make the Query and Filter Manager a powerful tool for managing complex database searches and alert filters.

Possible changes in query behavior after upgrading from a release prior to 5.3.0 to 5.3.0 or later – The 5.3.0 release introduced subtle differences in the new query builder from the builder in previous releases. Although all new queries that you create in 5.3.0 or later will automatically be in the new format, all queries that existed before you upgraded will remain in the legacy format. If you load an existing query into the query builder to look at it and run the query but do not alter the syntax by making any changes to a field, operator, or value, then the new query format will not be applied and the legacy format will be retained. However, if you edit and save an existing query after upgrading to 5.3.0, it will be converted into the new format. This conversion occurs whether you use the query builder in the Query and Filter Manager (**System | Saved Objects | Queries and Filters**) or a query builder on one of several feature pages throughout the Stellar Cyber interface. Because this might introduce some behavior changes that produce unexpected results, use the **Run** tool to first verify the query and make sure you're getting the type of results you expect before saving it.

When there are queries in the legacy format with rules that might produce different results if their syntax was converted into the new format, Stellar Cyber displays a notification about this above the queries table in the Query and Filter Manager. If you see this notification, view the flagged queries and use the **Run** option to test them individually before converting them to the new format.

Although the filter builder is similar to the query builder, the format of alert filters created in previous releases is not affected when upgraded to 5.3.0. Because case filters were introduced in 5.5.0, upgrades from earlier versions have no bearing on them.

Best Practices for Writing Efficient Queries

"Warning: Expensive Query Detected: This query is resource-intensive and may slow down the system. Consider optimizing or simplifying the query to improve efficiency."

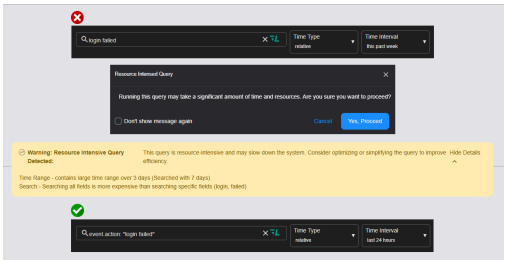
The above message appears when Stellar Cyber determines that a query is likely to require significant system resources due to how it’s constructed. While the platform doesn’t block the query, it provides this advisory to help you improve efficiency and avoid potential performance issues. Optimizing your query can reduce delays and improve responsiveness, especially in multi-tenant environments.

To begin, limit the time range of your query to focus only on what’s necessary. Large time spans—particularly those over several days—require more data to be scanned and processed, which can slow results. Narrowing the timeframe to a specific window, such as the

past 24 hours or a known incident window, helps the system return results more quickly and with less load.

Whenever possible, specify which field you are searching. A query like `login failed` scans across all indexed fields, consuming more resources than necessary. Whenever possible, specify which field you are searching. A query like login failed scans across all indexed fields, consuming more resources than necessary. Instead, use structured queries such as `event.action: "login failed"` to target the relevant data more precisely and efficiently.

The screen captures below show how to optimize a broad, time-intensive query by targeting specific fields and narrowing the time range.



Avoid wildcards unless absolutely required. Leading wildcards (e.g., *admin) are especially costly, as they force the system to scan entire indices for potential matches. Even contained or trailing wildcards (e.g., admin*, *admin*) can introduce unnecessary overhead. If a partial match is needed, try to structure it within a specific field and avoid generic patterns.

Finally, simplify any aggregations used in dashboards or query outputs. Queries with multiple layers of nested aggregations or several aggregation paths can rapidly increase complexity and execution time. Whenever possible, reduce the depth of aggregations and limit the number of grouped fields to those that are essential for analysis.

Use the Query Builder on Feature Pages

You can use the query builder in different ways that improve threat detection and operational efficiency. Use queries to define parameters that filter the data you retrieve from the database. Integrate them into automation rules to set conditions that determine when to perform automated tasks. Include them in charts to control the data that appears in dashboards. Additionally, you can save, reuse, and share queries among team members.

A version of the query builder that's in the Query and Filter Manager appears on pages throughout the Stellar Cyber interface wherever there's data that can be affected by search results:

- **Threat Hunting | Correlation Search |** for **New query** in the **Configure** section
- **Automation | Automation | Create** to add a new playbook or ("Edit this row" icon) to edit an existing playbook **| New Query**
- **Reporting | Reports | Filters | Queries |** ("Open Query Builder" icon) **| New Query**
- **System | Saved Objects | Charts | Create** to add a new chart or ("Edit this row" icon) to edit an existing chart **| New Query** on the **Query** step in the chart builder
- **Dashboards | PREDEFINED | XDR Kill Chain | Filters | Queries |** ("Open Query Builder" icon) **| New Query**

- **Alerts | View** for an Alert Type **|** ("More info" icon) for an alert event **| Actions | Add Alert Customization**

The query builder on these pages is a more compact version of the one in the Query and Filter Manager but contains many of the same options: a list of saved queries for selection, the ability to edit (but not delete) saved queries, and an option to create new ones. When you select a saved query, Stellar Cyber executes it immediately and updates the page with the query results.

The **None** query at the top of the list is a special case that removes any effect of a query on the search results. It's selected by default.

Create a New Query in the Query Builder on a Feature Page

To create a new query:

1. Navigate to one of the feature pages with a query builder and select **New Query**.

   The **Query Builder** dialog box appears.

2. Enter a name for the query.

   Special characters are not permitted in name fields for queries, watchlists, reports, or dashboards. Letters, underscores, spaces, dashes, numbers and periods are permitted.

3. If you are logged in to Stellar Cyber at the root level, choose an individual tenant or **All Tenants**. This sets the scope of its availability.

4. Choose a **Condition Type**: **AND**, **NOT**, or **OR**.

   These logical operators determine the relationship between multiple conditions within the group.

   For information about the different settings and options in a query, see **Query Builder Parameters**.

5. **Add Condition** and enter a field, operator, and value to define a simple query.

6. Add more your conditions, string conditions, and inner groups to the group as necessary.

7. To create a compound query, select **Add new group** and repeat the previous steps.

8. **Save** the query.

   Stellar Cyber saves the query for future use on the page where you created it, on all feature pages with query builders,a nd in the Query and Filter Manager.

Build a Query from Active Filters

When you create a search or alert filter during an investigation, you can convert the active filter settings into a query instead of rebuilding the same logic manually in Query Builder. This helps you save time, preserve investigation logic, and reuse the resulting query later in other areas of the Stellar Cyber Platform.

For example, you might apply filters on **Detections | Alerts** to narrow the results to a specific set of alerts that you want to investigate again later. Instead of recreating the same logic from scratch, you can open the filtered results in Query Builder and save the generated query for reuse. Similarly, you might use **Investigate | Threat Hunting** to refine a search until you isolate suspicious activity that you want to save as a repeatable query for later investigation, reporting, or dashboard use.

When you use this feature, Stellar Cyber opens Query Builder with the supported filters already populated. (The time range must be set manually after conversion because the Time Type and Time Interval settings at the top of the page are not included in the generated query.) You can then review the generated query, refine it if necessary, and save it for future use. This is especially useful when you want to turn ad hoc investigation work into a repeatable query that you can use again in dashboards, reports, and other features.

Review the generated query before you run or save it. On the **Detections | Alerts** page, the following page-specific filters are converted as raw query strings and require manual refinement after conversion to edit as structured conditions in Query Builder: **XDR Kill Chain**, **Source**, and **Tags**.

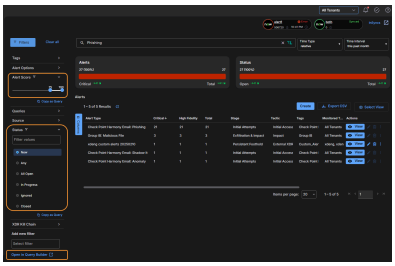
The **Open in Query Builder** option is available on the following pages:

- **Investigate | Threat Hunting**
- **Visualize | Predefined | Operational View**
- **Visualize | Predefined | Ingestion Dashboard**
- **Visualize | Predefined | Malware Sandbox**
- **Visualize | Predefined | Intrusion Signatures**
- **Visualize | Predefined | Service Visibility**
- **Visualize | Predefined | File Integrity Monitoring**
- **Detections | Alerts**

Build a Query from an Existing Search or Alert Filter

To build a query from active filters:

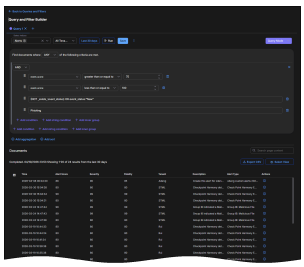
1. Navigate to a page where **Open in Query Builder** is available and then apply the search and filters that you want to reuse.
2. Confirm that the filtered results match what you want to carry over into Query Builder.
3. Select **Open in Query Builder**.



Stellar Cyber opens Query Builder and converts the supported filters into query conditions.

4. Set the time range in Query Builder to match that on the original page and then select **Run** to test the query.

   The time settings from the original page are not carried over into Query Builder, so you must set the time range manually before comparing results.

   Note that in the example screen captures shown here, the filtered search results on **Detections | Alerts** are grouped by Alert Type, while on Query Builder they're shown as individual alert instances. To compare the two views, add the values in the Total column on the Alerts page and compare that sum to the number of instances shown in Query Builder.

5. Review the generated query.

   Check that the generated conditions match the logic you intended to preserve.

6. Optionally refine the query.

   You can adjust conditions, groups, operators, and values before saving the query.

   For a query generated from the **Detections | Alerts** page, manually refine any generated conditions related to **XDR Kill Chain**, **Source**, or **Tags** as needed.

7. Save the query.

   Enter a meaningful name, optionally add notes, and then save it for future use.

8. Reuse the saved query where needed.

   After saving it, you can use the query again on feature pages with query builders and in **System | Saved Objects | Queries and Filters**.

Use the Query Builder in the Query and Filter Manager

The Query and Filter Manager (**System | Saved Objects | Queries and Filters**) includes a unified builder interface for creating queries, alert filters, and case filters. The builder operates in two interchangeable modes for queries and alert filters—allowing you to build, edit, and toggle between these types, since both use a shared Lucene syntax. This flexibility lets you view and adjust a query or alert filter in either context. In contrast, case filters use a distinct syntax specific to case creation logic, so when the builder is in case filter mode, it cannot be switched to query or alert filter mode. Regardless of the mode, the builder provides a graphical interface with options to select indices, define conditions, and create logical groups, generating the appropriate syntax automatically for you. (For instructions on using the query and filter builder to build alert filters, see **Use the Filter Builder in the Query and Filter Manager** below.)

Create a New Query in the Query and Filter Manager

To construct queries in the Query and Filter Manager:

1. Log in to the Stellar Cyber user interface (UI) and select **System | Saved Objects | Queries and Filters**.
2. Select **Create**.

   The Query and Filter Builder appears in Query mode. (To switch to Alert Filter mode, select **Query Mode | Alert Filter Mode**.)

   > The options used in query construction are also relevant when working with alert filters. Both involve defining conditions, applying logic, and using structured syntax to retrieve or filter data. For information about the different settings and options in a query, see **Query Builder Parameters**.

3. Choose the indices in which you want to run the query.

   If you create a query for a single index, you can set the time range up 1 month (**this past month**). If you choose two or more indices, the maximum time range can be up to 1 day (**last 24 hours**).

4. Set the scope of the query among Stellar Cyber tenants.

   About the Scope of Configured Objects

   When using the Stellar Cyber Platform, you can configure objects such as charts, correlations, and Automated Threat Hunting (ATH) rules that can be queried. In fact, even the queries that you create are objects. Root-level users can set the scope of these objects to be **All Tenants** or a specific tenant on the Stellar Cyber Platform. If a root user creates an object for **All Tenants**, then the object is available for every tenant to see and use, whereas if a root user creates an object for a specific tenant, then the object is available for just that one tenant. Partner-level users can create objects whose scope is a tenant in their tenant group, and tenant-level users can create objects whose scope is their own tenant, but only root-level users can create objects whose scope is **All Tenants**.

   When the object to be queried has a scope of **All Tenants**, then the scope of any query about this object must also be **All Tenants**. If the queried object has a scope of a specific tenant, then the query can be either **All Tenants** or the same specific tenant.

   When partner- and tenant-level users query objects that a root-level user created for **All Tenants**, they only see data that's within the sphere of their own tenant group (for partners) or tenant (for tenant users); they don't see data that belongs to any other tenant group or tenant. From the perspective of partners and tenant users, all the data retrieved from a query pertains only to their own tenant group or tenant, regardless of the scope used when creating and querying the object.

   In summary:

   - Root users can create objects with the scope of **All Tenants** or the scope of a specific tenant.
   - Partners can create objects with the scope of an individual tenant that's in their tenant group.

- Tenant users can create objects whose scope is their own tenant.
- Although partners and tenant users cannot create or modify objects whose scope is **All Tenants**, they can see and use these objects, as well as clone* them.
- If an object to be queried has **All Tenants** as its scope, then the scope of the query must also be **All Tenants**. It cannot be that of a single tenant.
- If an object to be queried has a scope of a specific tenant, then the scope of the query can be either **All Tenants** or the specified tenant.
- When partner- and tenant-level users query an object with a scope of **All Tenants**, they retrieve only data pertaining to their tenant group (for partners) or tenant (for tenant users).

  (* Partners and tenant users can clone an object that has a scope of **All Tenants** and save the object with the scope of an individual tenant. Then they can edit it and create queries for it.)

5. Set a time range for the data to query.

   If you're querying a single index, the time range can be up to **this past month**. If you're querying two or more indices, the time range can be up to the **last 24 hours**. Stellar Cyber uses the time range you set when you select **Run** to check if the query produces results as expected.

   The time range is not saved with the query itself; when you run a query later, Stellar Cyber automatically uses the currently active time range on the feature page where you run the query, up to the maximum allowed for querying a single index or multiple indices.

6. Add one or more conditions or string conditions.

   Conditions

   A condition is a specific criterion or set of criteria that data must meet to be included in the result set. Conditions are used to filter and refine data, ensuring that only relevant data is returned. Conditions are the building blocks of queries and filters, defining the criteria that data must meet to be returned (in response to queries) or excluded (in response to alert filters).

   The term *condition* is flexible and refers to an overall set of one or more subconditions as well as to the individual subconditions themselves. The overall condition can be composed of several subconditions grouped together using the logical operators **AND**, **OR**, or **NOT**. At the same time each subcondition is also considered a *condition* individually as well.

   - **Simple conditions**: A simple condition involves a single group in which the criterion or criteria of a query is defined as one or more conditions or string conditions.

- A condition consists of a field (dropdown list), an operator (dropdown list), and one or more values (list or text field). The operator defines how the field relates to the value with operators like **matches**, **greater than**, and **starts with**.

  To know which values to enter, refer to the data you have and choose or enter values accordingly.

  When the operator is **contains**, **does not contain**, **is**, or **is not**, you can select **+** to the right of the value field and enter additional values.

  When you see the **Aa** icon at the left of a condition, you can select it to toggle case sensitivity on and off. By default, new queries and existing queries created in releases prior to 5.3.0 and upgraded are case insensitive.

- A string condition defines a query with a free-form text string that uses Lucene query syntax. For example, enter the following to search for log entries where the `event_description` field contains the word `malware` or `virus` but excludes entries that include `test`:

  ```
  event_description: (malware OR virus) AND NOT test
  ```

  If a search string is a phrase with spaces in it, the order of the words in the phrase is respected. For example, if you enter `firewall security policy rules`, you will get results with these four words in exactly the same order. You won't get results with the same four words but in a different order such as `firewall policy security rules` or `security firewall policy rules`. Respecting word order in this way enables more predictive results.

The group-level operator (AND, OR, or NOT) determines the logical relationship between individual conditions within the group. The logical operator between groups is always OR.

- **Compound conditions**: A compound condition combines two or more groups to create a series of conditions. Like compound sentences, compound conditions are made up of multiple, independent simple conditions combined together into groups, each of which could stand alone. In Stellar Cyber, the logical relationship between groups in a compound condition is always OR.
- **Complex conditions**: A complex condition combines multiple simple conditions into a group and then nests if inside another condition, allowing you to create more nuanced queries by combining different criteria hierarchically. Complex conditions, like complex sentences, involve a primary condition with additional, subordinate conditions that add depth and complexity to the logic.

If you defined a query for the Alerts index, you can change it to an alert filter by selecting **Query Mode | Alert Filter Mode** and then saving it as an alert filter.

7. **Run** the query and check the results at the bottom of the page.

   If necessary, make changes to any of the settings and run it again. Keep making changes and testing the query until you're satisfied with the types of results you're getting.

8. Select **Save | Save As**, give the query a meaningful name, optionally add notes, and then **Save**.

   Stellar Cyber saves the query and displays it in the same tab with its new name.

9. To add another query or an alert filter, select the **Plus** icon (**+**) to the right of the active tab and select **New Query** or **New Alert Filter**.

   Stellar Cyber keeps the tab for the first query open, creates a new tab next to it, and switches to the new tab. You can leave both the first and second tabs open and switch back and forth between them as you work.

   You can have up to ten tabs open simultaneously. They can be all queries, all alert filters, or a mix of both.

Edit a Previously Saved Query

To edit a previously saved query:

1. Select a query from the **Saved Queries** list.

   If you start typing the query name, Stellar Cyber narrows the queries accordingly. When you select the query, the display changes to show the query in its structured form.

2. Select **Edit** to open the **Build a Query** dialog box.
3. You can change the name and description of the query.

   Special characters are not permitted in name fields for queries, watchlists, reports, or dashboards. Letters, underscores, spaces, dashes, numbers and periods are permitted.

4. You can change the tenant.
5. You can add, modify, and delete conditions.
6. **Save** the query when you're done with your changes.

If the query tool has a saved query selected and you select **Open Query Builder** to the right, the **Query Builder** dialog box opens with the selected query in edit mode.

Export and Import Queries

You can export queries from one Stellar Cyber instance and import them into another instance, or from one tenant into another tenant. This capability simplifies query sharing across instances and tenants, letting you share queries with other analysts and administrators

and also reuse queries that you defined for one situation in others without having to recreate them manually.

Exporting Queries

You can export queries in three ways, depending on your purpose:

- Export CSV (Entire Table) – Use this option when you need a spreadsheet view of all queries for reporting, inventory, or documentation purposes.
- Export Multiple Queries (JSON) 6.3.0 – Use this option when you want to migrate several queries at once or share them as a group with another tenant or analyst.
- Export Individual Query (JSON) 6.3.0 – Use this option when you need to migrate or share a single query.

Export CSV (Entire Table)

To export the entire Queries table in CSV format:

1. Select **Export CSV** at the top of the Queries table.

   A drop-down menu appears with the options **Selected columns** and **All columns**.
2. Select either option: **Selected columns** or **All columns**.

   Stellar Cyber exports the entire table as a CSV-formatted file.
3. Save the file to a directory on your local system.

   The exported file includes all query names, tenant associations, creation and update timestamps, and other metadata useful for administrative review or documentation.

Export Multiple Queries (JSON) 6.3.0

To export multiple queries in JSON format:

1. In the Queries table, select the check boxes next to the queries you want to export.

   The **Export Queries (**<number>**)** button becomes active above the table.
2. Select **Export Queries (**<number>**)**.

   Stellar Cyber exports the selected queries together in a single .json file, which includes any associated watchlists (formerly called *lookup lists*).

   These watchlists might be tenant-specific or shared across all tenants if created as **All Tenants** objects.
3. Save the file to a directory on your local system.

This method is ideal for migrating several queries between tenants or sharing a standardized query set with another analyst.

Export Individual Query (JSON) 6.3.0

To export an individual query:

1. In the **Actions** column of the query, select the **Export** icon (a down arrow).

Stellar Cyber exports the query as a single .json file containing its definition and metadata.

2. Save the file to a directory on your local system.

This option is most efficient when you need to migrate or share one query.

Sharing Queries

After exporting, you can share JSON files with other users or teams to promote consistency and collaboration across instances and tenants. You can distribute the files through shared repositories, internal documentation systems, or secure file transfer methods supported by your organization.

Sharing queries helps you and your fellow analysts and administrators standardize query logic, reduce redundant work, and ensure that tested search definitions are available to all relevant Stellar Cyber instances and tenants. Youf can also reuse queries you created for one situation in other Stellar Cyber instances and tenants that you manage, maintaining consistent analytic coverage across environments.

Importing Queries 6.3.0

You can import one or more previously exported queries into any Stellar Cyber instance or tenant.

To import queries:

1. Select the down arrow to the right of the **+ Create** button at the top of the Queries table.

   A drop-down menu appears.

2. Select **Import Query**.

   A file selection dialog box opens on your local system.

3. Navigate to the directory where the .json file containing the exported queries is stored and select it.

   Stellar Cyber imports the queries into the Query and Filter Manager.

   If any of the imported queries reference a watchlist (formerly called a lookup list), Stellar Cyber prompts you to choose how to handle the associated data.

   Watchlists are available under **SYSTEM | Saved Objects | Watchlists** in the 6.3.0 release. For details about creating and managing watchlists, see **Working with Watchlists** .

4. When importing a query that relies on a watchlist, consider whether the referenced watchlist exists in the destination tenant and whether it applies to all tenants or to a single tenant.

   Watchlists can be created either for a specific tenant or for **All Tenants**, in which case the watchlist is shared across all tenants in the same multi-tenant Stellar Cyber Platform.

5. To control how Stellar Cyber handles these dependencies, choose one of the following options:

    - **Overwrite** – Replace the existing watchlist in the destination instance or tenant with the version included in the import file.

      Use this option when the imported watchlist contains the most current or authoritative data and you want all queries that depend on it to use the imported values.

    - **Merge** – Combine the entries from the imported and existing watchlists, automatically removing duplicate entries.

      Use this option when both Stellar Cyber the source and destination maintain useful entries and you want to preserve data from each. This approach is effective for consolidating lists such as domain or IP address blocklists managed by administrators in different instances and tenants.

    - **Use Existing** – Retain the watchlist already present in the destination and ignore the one in the import file.

      Use this option when the watchlist in the destination is more accurate or locally maintained, or when you want to ensure that imported queries continue to operate against the established dataset in the destination tenant.

    Matching watchlists with identical names and identical contents are automatically preserved, with no action required.Role-based access controls continue to apply to imported content.

Delete a Previously Saved Query

To delete a previously saved query:

1. Select **System | Saved Objects | Queries and Filters**, and check the **In Use** column in the Queries table to see the number of places where the query you want to delete is being used.
2. Hover your cursor over the number to see a pop-up list of the features using the query.
3. Remove the query from use in these features and then return to the Queries table.

   When the **In Use** column has 0 entries, the **Delete** icon appears in the **Actions** column.
4. Select **Delete** and then select **Yes** to confirm the query deletion when prompted.

Query Builder Parameters

The following are details about the various parameters in the **query builder**.

Query Name

Choose a name that makes the query easy to find again later.



Special characters are not permitted in name fields for queries, watchlists, reports, or dashboards. Letters, underscores, spaces, dashes, numbers and periods are permitted.

Condition Type

The options are **AND**, **NOT**, **OR**. These boolean operators are applied when the new condition is entered and define how conditions relate to one another when the query is evaluated.

- **AND** returns only results that meet all of the specified conditions.
- **NOT** excludes results that meet the listed condition or group of conditions.
- **OR** returns results that meet any of the specified conditions.

At the top (group) level, the operator determines how the entire set of subconditions is combined, while within a nested group, it defines the logical relationship among the individual subconditions inside that group.

The condition type does not apply to the first condition in a new group. The first condition defines the starting point of the group, so a logical operator is needed only when you add a second condition or create an inner group.

In the Stellar Cyber interface, the selector wording differs slightly from the logical terms stored in the underlying query condition markup:

Value in the UI	Value in the Condition Markup
NONE	NOT
ALL	AND
ANY	OR

For example, if a query in the Query Builder is configured as—

> Find documents where **NONE** of the following criteria are met
>
> Field = assignee, Operator = is, Value = schmetterling

—the condition markup generated for this filter is:

```
{"conditions":
[{"caseSensitive":false,"field":"assignee","op":"==","type":"stri
ng","value":["schmetterling"]}],"modifier":"not"}
```

The difference is that the UI uses **NONE** in the interface, while the markup stores the logical equivalent **not**. This difference appears only in the top-level condition group; nested groups always use the standard logical operators (**AND**, **OR**, and **NOT**) directly.

Add Condition

Adds a new condition to the query. The condition includes a field, operator, and value.

- **Field**

  The field is the data object in the Interflow record to be tested by the condition. There can be a large number of objects in an Interflow record, and the drop-box provides some of them. You can also enter the field name.

See the Interflow overview page for more information on possible field selections.

- **Operator**

The operator defines the relationship between the field and the value. he following is a descriptive list of all the operators that Stellar Cyber supports.

Use `is` or `is not` only for exact full matches. Use `contains` or `does not contain` when the match can occur anywhere in a field.

Operators

1. is	11. greater than
2. is not	12. greater than or equal to
3. contains	13. less than
4. does not contain	14. less than or equal to
5. matches	15. in range
6. does not match	16. within
7. field exists	17. not within
8. field does not exist	18. is in lookup
9. starts with	19. is not in lookup
10. ends with	

1. is

Behavior: The `is` operator looks for an exact match of the value in a specified field.

Definition: Returns results when the field contains a value that exactly matches the specified search term. When you enter multiple search tokens (individual words and numbers), the search preserves their order and returns results only if they appear in the same order.

You can toggle case sensitivity on and off.

Field type: Text string, number, date, boolean

Example: To find a specific value in the `event_name` field in network logs:

```
event_name is Impossible Travel Anomaly
```

This query searches for events when the value in the `event_name` field is exactly `Impossible Travel Anomaly`, meaning events when a user appears to be in two locations within a timespan that would be physically infeasible, which suggests a potential security threat such as account compromise.

When you define a query with a time `field` and a `value` that includes milliseconds, it's not recommended to use `is` or `is not` as the `operator`. Use one of the following operators instead: `greater than`, `greater than or equal to`, `less than`, `less than or equal to`, or `in range`.

2. is not

Behavior: The `is not` operator excludes results when the field value matches the given term exactly.

Definition: Returns results when the field does not exactly match the specified value. When you enter multiple search tokens, the search preserves their order and returns results only if they appear in the same order.



You can toggle case sensitivity on and off.

Field type: Text string, number, date, boolean

Example: To find events that are not classified as impossible travel anomalies:

```
event_name is not Impossible Travel Anomaly
```

This query excludes any logs when the value in the `event_name` field is `Impossible Travel Anomaly` while returning events classified as other types of security incidents.



When you define a query with a time `field` and a `value` that includes milliseconds, it's not recommended to use `is` or `is not` as the `operator`. Use one of the following operators instead: `greater than`, `greater than or equal to`, `less than`, `less than or equal to`, or `in range`.

3. contains

Behavior: The `contains` operator looks for a match with the value in a specified field.

Definition: Returns results when a field contains the given value regardless of the exact order of the terms found in a full text search.



You can toggle case sensitivity on and off.

Field type: Text string

Example: To find events when the value in the `xdr_event.description` field contains the term `malware` anywhere within the text:

```
xdr_event.description contains malware
```

This query matches any log when the value of the `xdr_event.description` field contains the term `malware` at any position in the text, such as `malware detected` or possible `malware activity`.

4. does not contain

Behavior: The `does not contain` operator excludes results when the value of a specified field contains the specified value.

Definition: Returns results when a field does not include the given search term or terms as part of its value regardless of the exact order of the terms found in a full-

text search.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To exclude events when the `xdr_event.description` field contains the word `malware`:

```
xdr_event.description does not contain malware
```

This query finds logs when the value in the `xdr_event.description` field does not contain `malware` anywhere in the text.

5. matches

Behavior: The `matches` operator finds results that match a specific pattern in the value of a specified field.

Definition: Returns results when the field value matches a given pattern, which can be an exact pattern found in a full-text search; that is, the search might match phrases containing the search terms even if they are in a different order or separated by other words.

Field type: Text string

Example: To match all XDR event descriptions that include application and traffic:

```
xdr_event.description matches application traffic
```

This query finds logs when the value in the `xdr_event.description` field includes `application` and `traffic`, even when the tokens `application` and `traffic` are not adjacent as written as the value.

6. does not match

Behavior: The `does not match` operator excludes results that match a specific pattern.

Definition: Returns results when the field value does not match the specified pattern found in a full-text search; that is, the search might match phrases containing the search terms even if they are in a different order or separated by other words.

Field type: Text string

Example: To exclude events when the `xdr_event.description` field includes application and traffic, even when the tokens `application` and `traffic` are not adjacent as written as the value:

```
xdr_event.description does not match application traffic
```

This query returns all logs when the value of the `xdr_event.description` field does not include `application` and `traffic`.

7. field exists

Behavior: The `field exists` operator checks if a specific field is present. The field name must be an exact match. There is no value for this operator.

Definition: Returns results when the specified field is present (even if it's empty).

Field type: All field types

Example: To find all logs where the `event_name` field exists:

```
event_name field exists
```

This query returns all logs when the `event_name` field exists, meaning there is an event name associated with the logs.

8. field does not exist

Behavior: The `field does not exist` operator checks if a specific field is missing from a log. The field name must be an exact match. There is no value for this operator.

Definition: Returns results when the specified field does not exist.

Field type: All field types

Example: To find all logs where the `event_name` field does not exist:

```
event_name field does not exist
```

This query returns logs when the `event_name` field does not exist, meaning there is no event name associated with the logs.

9. starts with

Behavior: The `starts with` operator checks for a match at the beginning of the value for a specified field.

Definition: Returns results when the field value starts with the specified value.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To find all logs where the `event_name` field value starts with `port`:

```
event_name starts with port
```

This query returns all logs when the value of the `event_name` field starts with `port`, such as `port_scan` or `port_block`.

10. ends with

Behavior: The `ends with` operator checks for a match at the end of the value in a specified field.

Definition: Returns results when the field value ends with the specified value.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To find all logs where the `event_name` field value ends with `scan`:

```
event_name ends with scan
```

This query finds logs when the value of the `event_name` field ends with `scan`, such as `port_scan` or `network_scan`.

11. greater than

Behavior: The range query using the `greater than` operator checks for values greater than a specified value.

Definition: Returns results when the field value is greater than the given threshold.

Field type: Number, date

Example: To find critical events, which have a severity level of 75 or higher:

```
event_score is greater than 74
```

This query finds events when the value in the `event_score` field is greater than 74, capturing only critical events.

 Another way to perform the same kind of search using different Lucene query syntax is `event_score:{74 TO *}`. Both methods are valid in Lucene.

12. greater than or equal to

Behavior: The range query using the `greater than or equal to` operator checks for values greater than or equal to a specified value.

Definition: Returns results when the field value is greater than or equal to the given value.

Field type: Number, date

Example: To find events where the severity is greater than or equal to 75:

```
event_score is greater than or equal to 75
```

This query finds logs when the value in the `event_score` field is greater than or equal to 75, capturing only critical events.

 Another way to perform the same kind of search using different Lucene query syntax is `event_score:[75 TO *]`. Both methods are valid in Lucene.

13. less than

Behavior: The range query using the `less than` operator checks for values less than a specified value.

Definition: Returns results when the field value is numerically less than the given value.

Field type: Number, date

Example: To find events where the severity level is less than 25:

```
event_score is less than 25
```

This query finds events when the severity is less than 25, capturing alerts with a severity classified as “notice”.



Another way to perform the same kind of search using different Lucene query syntax is `event_score:{* TO 25}`. Both methods are valid in Lucene.

14. less than or equal to

Behavior: The range query using the `less than or equal to` operator checks for values less than or equal to a specified value.

Definition: Returns results when the field value is less than or equal to the given value.

Field type: Number, date

Example: To find events where the severity is less than or equal to 24:

```
event_score is less than or equal to 24
```

This query finds events when the severity is less than or equal to 24, capturing alerts with a severity classified as “notice”.



Another way to perform the same kind of search using different Lucene query syntax is `event_score:[* TO 24]`. Both methods are valid in Lucene.

15. in range

Behavior: The `in range` operator searches for values that fall within a specified range of numbers or dates.

Definition: Returns results when a field value falls between two specified values (inclusive), represented by `[a TO b]`, when `a` is the lower bound and `b` is the upper bound. The query matches if the value is greater than or equal to `a` and less than or equal to `b`.

Field type: Number, date

Example: To find events where the `event_score` falls between 50 and 74 inclusive, which categorizes them as major alerts:

```
event_score is in range (from) 50 (to) 74
```

This query matches logs when the `event_score` is between 50 and 74, including 50 and 74.

16. within

Behavior: The `within` operator looks for IP addresses that fall within a specified IP address range or netmask.

Definition: Returns results when an IP address is within the given range or subnet defined by an IP address and netmask. The query matches IP addresses that are within the specified network block.

Field type: IP addresses and IP addresses with netmasks

Example: To find events where the `srcip` is within the subnet 192.168.1.0/24:

```
srcip is within 192.168.1.0/24
```

This query matches logs when the source IP address falls within the 192.168.1.0/24 subnet, which covers IP addresses from 192.168.1.0 to 192.168.1.255.

17. not within

Behavior: The `not within` operator excludes IP addresses that fall within a specified IP address range or netmask.

Definition: Returns results when an IP address is not within the given range or subnet. The query excludes any IP addresses within the defined network block.

Field type: IP addresses and IP addresses with netmasks

Example: To exclude events where the `dstip` is within the subnet 10.0.0.0/8:

```
dstip is not within 10.0.0.0/8
```

This query excludes logs when the destination IP address falls within the 10.0.0.0/8 subnet, which covers IP addresses from 10.0.0.0 to

10.255.255.255.

18. is in lookup

Behavior: The `is in lookup` operator checks whether the value of a specified field matches any entry in a predefined lookup list.

Definition: Returns results when the value of the given field is found in a lookup list. The query matches if the field value exists in the list.

You can toggle case sensitivity on and off.

Field type: Text string, number, date, IP address

Example: To find events where the `srcip` is in a lookup list of known malicious IP addresses:

```
srcip is in lookup malicious_ips_lookup
```

This query matches logs when the source IP address appears in the `malicious_ips_lookup` list.

19. is not in lookup

Behavior: The `is not in lookup` operator excludes results when the value of a specified field matches an entry in a predefined lookup list.

Definition: Returns results when the value of the given field is not found in a lookup list. The query excludes any logs when the field value exists in the list.


You can toggle case sensitivity on and off.

Field type: Text string, number, date, IP address

Example: To exclude events where the `dstip` is in a lookup list of trusted IP addresses:

```
dstip is not in lookup trusted_ips_lookup
```

This query excludes logs when the destination IP address appears in the `trusted_ips_lookup` list.

- **Value**

  The value the condition field will be tested against.

When there are multiple conditions, you can drag and drop them to rearrange their order.


Reordering conditions inner groups within their outer group doesn't affect their logical relationship or the results of a query or filter. It simply lets you organize multiple conditions and inner groups within a group as you prefer, such as placing all the conditions at the top and all the inner groups at the bottom.

The following are some points to know when creating conditions:

- When the **Aa** icon appears at the left of a condition, you can toggle context-sensitivity on and off. By default, case-sensitivity is off. When case-sensitivity is on, there will only be exact matches to the case of a value. For example, if case sensitivity is off and the value is `test`, all of these are matches: `test`, `Test`, and `TEST`. If case sensitivity is on, then only `test` matches `test`.

  Enable case sensitivity when you need to distinguish values based on capitalization; for example, when differentiating `script.sh` from `Sript.sh` or matching case-sensitive credentials. Leave it off for broader matching in paths, keywords, or log messages where case variation is common.

- To know which field name to choose from the Field dropdown list, check the field names that show up in event details such as those at **Alerts | View** for an Alert Type | ℹ ("More info" icon) for an alert event **| JSON**. Then start typing the field name into the text box. Stellar Cyber uses a predictive search to dynamically narrow down the

available options based on your input until you spot the one you want. You can also supplement the predictive search by using the scroll bar to find the field name in the alphabetically ordered list.

> To create a condition with a field name that isn't in the Field dropdown list, type in the complete field name. Stellar Cyber displays a new dropdown list between Field and Operator with the following choices of value types: **boolean**, **date**, **ip**, **number**, and **string**. Choose the value type, then the operator, and finally enter a value.

- The values are validated as they are entered. An error message appears if, for example, the value should be an IPv4 address (such as `10.1.1.1`) but it was not formatted in dotted decimal notation.
- When the operator is **is**, **is not**, **contains**, or **does not contain**, the ✚ ("Plus") icon appears, and you can enter multiple values. The relationship between the values is **OR**.
- When the Value field doesn't present a calendar and clock for you to select a date and time, manually enter them in UNIX Epoch format, in seconds.
- IP addresses with subnet masks are supported.
- There is no **Value** for the operators **field exists** and **field does not exist**. Simply choose the field that you want to filter alerts for if it exists in an Interflow record or does not exist.
- The **is in watchlist** and **is not in watchlist** operators let your condition reference a reusable, dynamic list of values. Instead of configuring the same list of values over multiple filters, you can instead reference a watchlist (see **Working with Watchlists**). If you want to add or remove values, you just need to modify the watchlist instead of modifying each filter.
- Invalid conditions—for example, duplicate conditions—are underlined in red and an error icon appears at the end of the condition. Select the error icon to display an error message. Either select **Remove duplicate** to allow Stellar Cyber to remove the duplicate condition automatically or select **Dismiss** to correct the duplicate condition yourself.

When adding a condition to query for files or executable paths, follow these guidelines to ensure expected results:

- Do not use quotation marks around path values. Quoted strings are unnecessary and might interfere with search behavior.
- Use `is` for exact, complete values:

  ✓`process_path is C:\Windows\System32\conhost.exe` (exact match)

  ×`process_path is \System32\conhost.exe` (not an exact match)
- Use `contains` for partial matches where you only need to find a substring within a field or path:

  ✓`process_path contains \System32\conhost.exe` (partial match)

- Do not escape slashes. Queries interpret slashes literally and do not require escaping.

Add String Condition

A string condition consists of a free-form text string using Lucene query string syntax.

Add Inner Group

More complex queries might require inner groups. Inner groups are multiple conditions evaluated as a unit. They are equivalent to parenthetical terms in mathematics.

Selecting this adds an inner (or *child*) group within the outer (or *parent*) group, with its own set of conditions.

There can be a maximum of ten layers of nested inner groups in a query + the outer group that encompasses all the inner groups.

Remove This Group

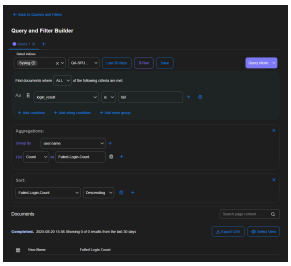
This button appears inside an inner group. Selecting this removes the inner group.

Use Aggregations in the Query Builder

The Query Builder supports grouping and aggregation functions so that you can summarize and analyze data beyond individual events.

- Grouping organizes results by one or more fields. For example, you can group login events by user or network flows by `src_ip` and `dst_ip`.
- Aggregations calculate summary values within each group. Supported functions include `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, and `DISTINCT COUNT`. These provide quick insights such as how many events occurred, the total data transferred, or the average response time.

Example: To identify accounts with the most failed login attempts, you can group events by user and count the number of failed logins for each account.



The result is a concise table showing each user and their number of failed login events, which highlights accounts with unusual activity.

Other ways to use aggregations include summing outbound bytes per user to detect potential data exfiltration, calculating average response time per service to monitor performance, and finding the minimum and maximum session duration for VPN users.

Limitations

- Aggregations and groupings are supported only in Query Builder. They are not applied in dashboards, reports, or embedded query builders in other features.

- Results are limited to the last 24 hours.
- Groupings are subject to a 10,000-bucket ceiling; queries that exceed this might display partial results without warning.

Remove

This button appears to the right of the condition parameter. Selecting it removes the condition.

Using Alert Filters and the Filter Builder

Alert filters exclude events that don't interest you so that you only receive alerts about those events on which you want to focus. Because Stellar Cyber suppresses filtered alerts, they don't appear in the **Alerts** index. You can create alert filters in the Queries and Filters Manager at **System | Saved Objects | Queries and Filters** or from alert event details: Select **Detections | Alerts | View** for an Alert Type, select **|** ("More info" icon) for an alert event, and then select **| Actions | Add Alert Customization**.

- **Use the Filter Builder in the Query and Filter Manager**
- **Use the Filter Builder in Alert Event Details**
- **Add Conditions and Groups**
  - **Alert Filter Parameters: Condition Type**
  - **Add a Condition**
  - **Add an Inner Group and a New Group**
  - **Keep Track of Per-Tenant Alert Filter Statistics**
- **Set an Expiration Period**
- **Actions**
- **Alert Filtering Examples**
- **Alert Filter Editability**

Use the Filter Builder in the Query and Filter Manager

To create an alert filter in the Query and Filter Manager:

1. Select **System | Saved Objects | Queries and Filters** and on the **Alert Filters** tab select **Create**.

   The query and filter builder appears in Alert Filter mode. (To switch to Query mode, select **Alert Filter Mode | Query Mode**.)

   The options used in alert filter construction are also relevant when working with queries. Both involve defining conditions, applying logic, and using structured syntax to filter or retrieve data.

2. If you are logged in to Stellar Cyber at the root level, choose one or more individual tenants or **All Tenants**, choose one or more tenant groups, or choose one or more detection profiles.

   You choice sets the scope of the alert filter.

You can choose a combination of tenants and tenant groups in the same alert filter. However, cannot combine tenants or tenant groups with detection profiles in the same alert filter.

Scope Alert Filters by Detection Profile

When you're logged in with root user access, you can scope an alert filter to one or more detection profiles. When you choose a detection profile, the alert filter applies to alerts from all tenants currently assigned to that profile. This scope is dynamic. If tenant membership in the detection profile changes in Detection Management, the alert filter scope updates automatically after the next refresh cycle. Changes can take up to 30 minutes to propagate.

About Share Scope

When you're logged in with root user access, you can set the scope of an alert filter to be **All Tenants**, one or more tenant groups, or one or more tenants. If you create an alert filter for **All Tenants**, then it's available for every tenant to see and use, and if you create an alert filter for specific tenant groups or tenants, then it's available for the tenants in the specified groups or the individually specified tenants. If you are logged in with partner-level access, you can create alert filters whose scope is your tenant group or one or more tenants in your group. If you are logged in with tenant-level access, you can create an alert filter whose scope is your own tenant.

Users at lower scopes can see and use alert filters that apply to their tenants when these filters were created by users at higher levels. Partner- and tenant-level users see alert filters that root-level users created for **All Tenants** and for tenant groups and tenants if the filters apply to them. Tenant-level users see alert filters that partners created for their tenant and the tenant group to which they belong.

When partner- and tenant-level users use alert filters that a root-level user created for **All Tenants**, one ore more tenant groups, or multiple tenants, they only filter alerts that are within the sphere of their own tenant group (for partners) or tenant (for tenant users); they don’t filter alerts that belong to any other tenant group or tenant. From the perspective of partners and tenant users, all alert filters pertain only to alerts within their own tenant group or tenant, regardless of the scope used when creating them.

In summary:

- Root users can create alert filters with the scope of **All Tenants**, one or more tenant groups, or one or more individual tenants.
- Partners can create alert filters with the scope of one or more tenants in their tenant group.
- Tenant users can create alert filters whose scope is their own tenant.
- Although partners and tenant users cannot create or modify alert filters whose scope is **All Tenants**, they can see and use them, as well as clone* them.

- When partner- and tenant-level users use alert filters with a scope of “All Tenants”, they only filter alerts pertaining to their tenant group (for partners) or tenant (for tenant users).

  (* Partners and tenant users can clone an alert filter that has a scope of **All Tenants** and save it with the scope of an individual tenant. Then they can customize the cloned alert filter to better suit their needs.)

3. Set a time range for the alerts to filter.

   The time range can be up to **this past month**.

4. Set the operator (**ALL**, **ANY**, or **NONE**) to determine the relationship between conditions in the group.

5. Select **Add condition**.

   For details about defining conditions and adding more conditions, inner groups, and new groups of conditions, see Add Conditions and Groups and the other sections later in this topic.

   You can change an alert filter to a query by selecting **Alert Filter Mode | Query Mode** and then saving it as a query.

6. **Test** the alert filter to verify it's configured correctly.

   When you select **Test**, Stellar Cyber evaluates your alert filter against alerts detected in the past 24 hours that match the specified conditions. It displays a list of alerts that would have been excluded if the filter had been in place at the time these alerts were detected. This lets you quickly assess whether the filter targets the correct types of alerts. If the results are not as expected, adjust the filter conditions and run the test again. Repeat as needed until you're confident the filter is excluding the intended alerts.

   Important: When you use the **Test** button in the Alert Filter Builder, Stellar Cyber performs a partial or fuzzy match of your filter conditions. This preview helps you quickly see what might match. However, during live filter execution, the system evaluates filter conditions strictly against the field values stored in the Interflow records. The production alert filtering engine compares the filter condition values exactly with the JSON fields in these records. Therefore, to ensure your alert filter functions as intended, do not include wildcard characters such as asterisks (`*`) or escape sequences in alert filter values because the system treats these characters literally rather than as pattern matches. Always specify the filter value exactly as it appears in the Interflow JSON.

   For example, consider how the following filter condition behaves differently in testing compared to actual alert filtering:

   - If you define an alert filter with `xdr_event.description contains *.oraclevcn.com`, the **Test** button might show a match for records like `www.oraclevcn.com`.
   - In production filtering, this condition matches only a record containing exactly `*.oraclevcn.com`. Therefore, do not use wildcard characters in

alert filter conditions because the actual results will not match what the **Test** button previ leads you to expect.

Test results are provisional and filters should be verified once deployed.

7. Select **Save**, give the alert filter a meaningful name, optionally add a note about it for future reference, and then **Save**.

   Stellar Cyber saves the alert filter and displays it in the same tab with its new name.

   As soon as you save an alert filter, Stellar Cyber immediately applies it to alerts. You don't need to apply alert filters manually; upon their creation, Stellar Cyber applies them automatically.

   To edit the name or notes of an alert filter, select the three-dot icon to the right of the **Save** button and choose **Edit Name and Notes**.

8. To add another alert filter, a query, or a case filter, select the **Plus** icon (**+**) to the right of the active tab and select **New Alert Filter**, **New Query**, or **New Case Filter**.

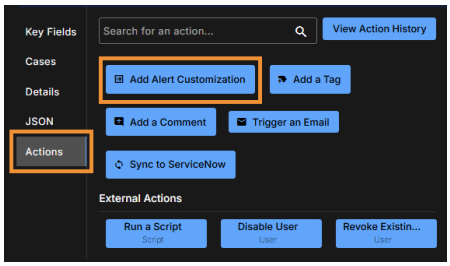
   Stellar Cyber keeps the tab for the first alert filter open, creates a new tab next to it, and switches to the new tab. You can leave both the first and second tabs open and switch back and forth between them as you work.

   You can have up to ten tabs open simultaneously. They can be all alert filters, all queries, all case filters, or a mix of all three types.

Use the Filter Builder in Alert Event Details

To create an alert filter from the alert event details:

1. Select **Detections | Alerts | View** for an Alert Type, select ("More info" icon) for an alert event, and then select **Actions | Add Alert Customization**.



   The **Alert Customizations** dialog box appears with fields pre-populated based on the selected alert.

2. If you are logged in to Stellar Cyber at the root level, choose an individual tenant or **All Tenants**. This sets the scope of its availability.

3. Modify the configuration if necessary.

   For details on defining a condition and adding more conditions, inner groups, and new groups, see the next section, Add Conditions and Groups.

4. **Test** the alert filter settings and then **Save** the configuration.

5. Enter a **Name** for the alert filter, optionally enter a note for future reference, and the **Save** it.

   The name can be up to 100 characters including spaces. The note can be up to 500 characters, including spaces.

   As soon as you save the alert filter, Stellar Cyber immediately applies it to alerts. You don't need to apply alert filters manually; upon their creation, Stellar Cyber applies them automatically.

Add Conditions and Groups

In Stellar Cyber, queries and alert filters are constructed using Lucene syntax (conditions and groups) that define criteria for matching data. A group consists of one or more conditions and can also contain inner groups to build complex logical expressions. When there are multiple outer groups, each one operates independently and is connected by an "OR" relationship, meaning that if the conditions defined in any outer group are true, the overall query or alert filter condition is satisfied. Within each group, the logical relationships among individual conditions and inner groups can be defined using "AND," "OR," or "NOT" to specify how the conditions should interact. Conditions within a group using "AND" must all be true for the group to match, "OR" allows any condition within the group to trigger a match, while "NOT" excludes any condition within a group from triggering a match. By combining these elements, you can create alert filters that suppress only the alerts matching your criteria so you can focus on more relevant security events.

When building an alert filter, you can add conditions, inner groups, and new groups. There is no maximum number of components that you add to an alert filter. However, the maximum number of layers of nested inner groups is ten (not counting the outer group that encompasses the nest of ten inner groups within it).

Alert Filter Parameters: Condition Type

The options are **ALL**, **ANY**, and **NONE**. These boolean operators are applied when the new condition is entered and define how conditions relate to one another when the filter is evaluated.

- **ALL** returns only alerts that meet all of the specified conditions.
- **ANY** returns alerts that meet any of the specified conditions.
- **NONE** excludes alerts that meet the listed condition or group of conditions.

At the top (group) level, the operator determines how the entire set of subconditions is combined, while within a nested group, it defines the logical relationship among the individual subconditions inside that group.

The condition type does not apply to the first condition in a new group. The first condition defines the starting point of the group, so a logical operator is needed only when you add a second condition or create an inner group.

In the Stellar Cyber interface, the selector wording differs slightly from the logical terms stored in the underlying alert filter condition markup:

Value in the UI	Value in the Condition Markup
ALL	AND
ANY	OR
NONE	NOT

For example, if an alert filter is configured as—

> Find documents where **NONE** of the following criteria are met
>
> Field = assignee, Operator = is, Value = schmetterling

—the condition markup generated for this filter is:

```
{"conditions":
[{"caseSensitive":false,"field":"assignee","op":"==","type":"stri
ng","value":["schmetterling"]}],"modifier":"not"}
```

The difference is that the UI uses **NONE**, while the markup stores the logical equivalent **not**. This difference appears only in the top-level condition group; nested (inner) groups always use the standard logical operators (**AND**, **OR**, and **NOT**) directly.

Add a Condition

A condition is a specific criterion or set of criteria that data must meet to be included in the result set.

To add a condition to a group, select **Add condition** and then define its components: field, operator, and value (in some cases multiple values). You can add as many conditions as you like. The operator defines the relationship between the field and the value. The following is a descriptive list of all the operators that Stellar Cyber supports.



Use `is` or `is not` only for exact full matches. Use `contains` or `does not contain` when the match can occur anywhere in a field.

Operators

1. is	11. greater than
2. is not	12. greater than or equal to
3. contains	13. less than
4. does not contain	14. less than or equal to
5. matches	15. in range
6. does not match	16. within
7. field exists	17. not within
8. field does not exist	18. is in lookup
9. starts with	19. is not in lookup
10. ends with	

1. is

Behavior: The `is` operator looks for an exact match of the value in a specified field.

Definition: Returns results when the field contains a value that exactly matches the specified search term. When you enter multiple search tokens (individual words and numbers), the search preserves their order and returns results only if they appear in the same order.



You can toggle case sensitivity on and off.

Field type: Text string, number, date, boolean

Example: To find a specific value in the `event_name` field in network logs:

```
event_name is Impossible Travel Anomaly
```

This query searches for events when the value in the `event_name` field is exactly `Impossible Travel Anomaly`, meaning events when a user appears to be in two locations within a timespan that would be physically infeasible, which suggests a potential security threat such as account compromise.



When you define a query with a time `field` and a `value` that includes milliseconds, it's not recommended to use `is` or `is not` as the `operator`. Use one of the following operators instead: `greater than`, `greater than or equal to`, `less than`, `less than or equal to`, or `in range`.

2. is not

Behavior: The `is not` operator excludes results when the field value matches the given term exactly.

Definition: Returns results when the field does not exactly match the specified value. When you enter multiple search tokens, the search preserves their order and returns results only if they appear in the same order.



You can toggle case sensitivity on and off.

Field type: Text string, number, date, boolean

Example: To find events that are not classified as impossible travel anomalies:

```
event_name is not Impossible Travel Anomaly
```

This query excludes any logs when the value in the `event_name` field is `Impossible Travel Anomaly` while returning events classified as other types of security incidents.



When you define a query with a time `field` and a `value` that includes milliseconds, it's not recommended to use `is` or `is not` as the `operator`. Use one of the following operators instead: `greater than`, `greater than or equal to`, `less than`, `less than or equal to`, or `in range`.

3. contains

Behavior: The `contains` operator looks for a match with the value in a specified field.

Definition: Returns results when a field contains the given value regardless of the exact order of the terms found in a full text search.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To find events when the value in the `xdr_event.description` field contains the term `malware` anywhere within the text:

```
xdr_event.description contains malware
```

This query matches any log when the value of the `xdr_event.description` field contains the term `malware` at any position in the text, such as `malware detected` or possible `malware activity`.

4. does not contain

Behavior: The `does not contain` operator excludes results when the value of a specified field contains the specified value.

Definition: Returns results when a field does not include the given search term or terms as part of its value regardless of the exact order of the terms found in a full-text search.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To exclude events when the `xdr_event.description` field contains the word `malware`:

```
xdr_event.description does not contain malware
```

This query finds logs when the value in the `xdr_event.description` field does not contain `malware` anywhere in the text.

5. matches

Behavior: The `matches` operator finds results that match a specific pattern in the value of a specified field.

Definition: Returns results when the field value matches a given pattern, which can be an exact pattern found in a full-text search; that is, the search might match phrases containing the search terms even if they are in a different order or separated by other words.

Field type: Text string

Example: To match all XDR event descriptions that include application and traffic:

```
xdr_event.description matches application traffic
```

This query finds logs when the value in the `xdr_event.description` field includes `application` and `traffic`, even when the tokens `application` and `traffic` are not

adjacent as written as the value.



6. does not match

Behavior: The `does not match` operator excludes results that match a specific pattern.

Definition: Returns results when the field value does not match the specified pattern found in a full-text search; that is, the search might match phrases containing the search terms even if they are in a different order or separated by other words.

Field type: Text string

Example: To exclude events when the `xdr_event.description` field includes application and traffic, even when the tokens `application` and `traffic` are not adjacent as written as the value:

```
xdr_event.description does not match application traffic
```

This query returns all logs when the value of the `xdr_event.description` field does not include `application` and `traffic`.

7. field exists

Behavior: The `field exists` operator checks if a specific field is present. The field name must be an exact match. There is no value for this operator.

Definition: Returns results when the specified field is present (even if it's empty).

Field type: All field types

Example: To find all logs where the `event_name` field exists:

```
event_name field exists
```

This query returns all logs when the `event_name` field exists, meaning there is an event name associated with the logs.

8. field does not exist

Behavior: The `field does not exist` operator checks if a specific field is missing from a log. The field name must be an exact match. There is no value for this operator.

Definition: Returns results when the specified field does not exist.

Field type: All field types

Example: To find all logs where the `event_name` field does not exist:

```
event_name field does not exist
```

This query returns logs when the `event_name` field does not exist, meaning there is no event name associated with the logs.

9. starts with

Behavior: The `starts with` operator checks for a match at the beginning of the value for a specified field.

Definition: Returns results when the field value starts with the specified value.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To find all logs where the `event_name` field value starts with `port`:

```
event_name starts with port
```

This query returns all logs when the value of the `event_name` field starts with `port`, such as `port_scan` or `port_block`.

10. ends with

Behavior: The `ends with` operator checks for a match at the end of the value in a specified field.

Definition: Returns results when the field value ends with the specified value.

 You can toggle case sensitivity on and off.

Field type: Text string

Example: To find all logs where the `event_name` field value ends with `scan`:

```
event_name ends with scan
```

This query finds logs when the value of the `event_name` field ends with `scan`, such as `port_scan` or `network_scan`.

11. greater than

Behavior: The range query using the `greater than` operator checks for values greater than a specified value.

Definition: Returns results when the field value is greater than the given threshold.

Field type: Number, date

Example: To find critical events, which have a severity level of 75 or higher:

```
event_score is greater than 74
```

This query finds events when the value in the `event_score` field is greater than 74, capturing only critical events.

 Another way to perform the same kind of search using different Lucene query syntax is `event_score:{74 TO *}`. Both methods are valid in Lucene.

12. greater than or equal to

Behavior: The range query using the `greater than or equal to` operator checks for values greater than or equal to a specified value.

Definition: Returns results when the field value is greater than or equal to the given value.

Field type: Number, date

Example: To find events where the severity is greater than or equal to 75:

```
event_score is greater than or equal to 75
```

This query finds logs when the value in the `event_score` field is greater than or equal to 75, capturing only critical events.



Another way to perform the same kind of search using different Lucene query syntax is `event_score:[75 TO *]`. Both methods are valid in Lucene.

13. less than

Behavior: The range query using the `less than` operator checks for values less than a specified value.

Definition: Returns results when the field value is numerically less than the given value.

Field type: Number, date

Example: To find events where the severity level is less than 25:

```
event_score is less than 25
```

This query finds events when the severity is less than 25, capturing alerts with a severity classified as “notice”.



Another way to perform the same kind of search using different Lucene query syntax is `event_score:{* TO 25}`. Both methods are valid in Lucene.

14. less than or equal to

Behavior: The range query using the `less than or equal to` operator checks for values less than or equal to a specified value.

Definition: Returns results when the field value is less than or equal to the given value.

Field type: Number, date

Example: To find events where the severity is less than or equal to 24:

```
event_score is less than or equal to 24
```

This query finds events when the severity is less than or equal to 24, capturing alerts with a severity classified as “notice”.



Another way to perform the same kind of search using different Lucene query syntax is `event_score:[* TO 24]`. Both methods are valid in Lucene.

15. in range

Behavior: The `in range` ator searches for values that fall within a specified range of numbers or dates.

Definition: Returns results when a field value falls between two specified values (inclusive), represented by `[a TO b]`, when `a` is the lower bound and `b` is the upper bound. The query matches if the value is greater than or equal to `a` and less than or equal to `b`.

Field type: Number, date

Example: To find events where the `event_score` falls between 50 and 74 inclusive, which categorizes them as major alerts:

```
event_score is in range (from) 50 (to) 74
```

This query matches logs when the `event_score` is between 50 and 74, including 50 and 74.

16. within

Behavior: The `within` operator looks for IP addresses that fall within a specified IP address range or netmask.

Definition: Returns results when an IP address is within the given range or subnet defined by an IP address and netmask. The query matches IP addresses that are within the specified network block.

Field type: IP addresses and IP addresses with netmasks

Example: To find events where the `srcip` is within the subnet 192.168.1.0/24:

```
srcip is within 192.168.1.0/24
```

This query matches logs when the source IP address falls within the 192.168.1.0/24 subnet, which covers IP addresses from 192.168.1.0 to 192.168.1.255.

17. not within

Behavior: The `not within` operator excludes IP addresses that fall within a specified IP address range or netmask.

Definition: Returns results when an IP address is not within the given range or subnet. The query excludes any IP addresses within the defined network block.

Field type: IP addresses and IP addresses with netmasks

Example: To exclude events where the `dstip` is within the subnet 10.0.0.0/8:

```
dstip is not within 10.0.0.0/8
```

This query excludes logs when the destination IP address falls within the 10.0.0.0/8 subnet, which covers IP addresses from 10.0.0.0 to

10.255.255.255.

18. is in lookup

Behavior: The `is in lookup` operator checks whether the value of a specified field matches any entry in a predefined lookup list.

Definition: Returns results when the value of the given field is found in a lookup list. The query matches if the field value exists in the list.



You can toggle case sensitivity on and off.

Field type: Text string, number, date, IP address

Example: To find events where the `srcip` is in a lookup list of known malicious IP addresses:

```
srcip is in lookup malicious_ips_lookup
```

This query matches logs when the source IP address appears in the `malicious_ips_lookup` list.

19. is not in lookup

Behavior: The `is not in lookup` operator excludes results when the value of a specified field matches an entry in a predefined lookup list.

Definition: Returns results when the value of the given field is not found in a lookup list. The query excludes any logs when the field value exists in the list.



You can toggle case sensitivity on and off.

Field type: Text string, number, date, IP address

Example: To exclude events where the `dstip` is in a lookup list of trusted IP addresses:

```
dstip is not in lookup trusted_ips_lookup
```

This query excludes logs when the destination IP address appears in the `trusted_ips_lookup` list.

When there are multiple conditions, you can drag and drop them to rearrange their order.



Reordering conditions inner groups within their outer group doesn't affect their logical relationship or the results of a query or filter. It simply lets you organize multiple conditions and inner groups within a group as you prefer, such as placing all the conditions at the top and all the inner groups at the bottom.

The following are some points to know when creating conditions:

- When the **Aa** icon appears at the left of a condition, you can toggle context-sensitivity on and off. By default, case-sensitivity is off. When case-sensitivity is on, there will only be exact matches to the case of a value. For example, if case sensitivity is off and the value is `test`, all of these are matches: `test`, `Test`, and `TEST`. If case sensitivity is on, then only `test` matches `test`.

Enable case sensitivity when you need to distinguish values based on capitalization; for example, when differentiating `script.sh` from `Sript.sh` or matching case-sensitive credentials. Leave it off for broader matching in paths, keywords, or log messages where case variation is common.

- To know which field name to choose from the Field dropdown list, check the field names that show up in event details such as those at **Alerts | View** for an Alert Type **|** ("More info" icon) for an alert event **| JSON**. Then start typing the field name into the text box. Stellar Cyber uses a predictive search to dynamically narrow down the available options based on your input until you spot the one you want. You can also supplement the predictive search by using the scroll bar to find the field name in the alphabetically ordered list.

  To create a condition with a field name that isn't in the Field dropdown list, type in the complete field name. Stellar Cyber displays a new dropdown list between Field and Operator with the following choices of value types: **boolean**, **date**, **ip**, **number**, and **string**. Choose the value type, then the operator, and finally enter a value.

- The values are validated as they are entered. An error message appears if, for example, the value should be an IPv4 address (such as `10.1.1.1`) but it was not formatted in dotted decimal notation.
- When the operator is **is**, **is not**, **contains**, or **does not contain**, the + ("Plus") icon appears, and you can enter multiple values. The relationship between the values is **OR**.
- When the Value field doesn't present a calendar and clock for you to select a date and time, manually enter them in UNIX Epoch format, in seconds.
- Do not escape slashes. The queries in alert filters interpret slashes literally and do not require escaping.
- IP addresses with subnet masks are supported.
- There is no **Value** for the operators **field exists** and **field does not exist**. Simply choose the field that you want to filter alerts for if it exists in an Interflow record or does not exist.
- The **is in watchlist** and **is not in watchlist** operators let your condition reference a reusable, dynamic list of values. Instead of configuring the same list of values over multiple filters, you can instead reference a watchlist (see **Working with Watchlists**). If you want to add or remove values, you just need to modify the watchlist instead of modifying each filter.
- Invalid conditions—for example, duplicate conditions—are underlined in red and an error icon appears at the end of the condition. Select the error icon to display an error message. Either select **Remove duplicate** to allow Stellar Cyber to remove the duplicate condition automatically or select **Dismiss** to correct the duplicate condition yourself.

- If there are two or more alert filters with the same condition, such as a filter for **All Tenants** and other filters for specific tenants, Stellar Cyber applies the **All Tenants** filter first, then the specific tenant filters. This tenant override behavior allows an MSSP admin to set up some general settings for **All Tenants**, then fine-tune with more refined settings for each tenant.

Add an Inner Group and a New Group

To add an inner group to a group, select **Add inner group**. Once you do, the original group becomes the outer group. You can add up (up to 10 inner groups). This lets you nest conditions, which allows for more nuanced filtering by combining different criteria.

Use the following group operators to define the logical relationship between conditions within a group—the conditions within an outer group and the conditions within an inner group:

- **AND** – retrieves results that satisfy all the conditions
- **OR** – retrieves results that satisfy at least one of the conditions
- **NOT** – excludes results that satisfy at least one of the conditions

Select **Add new group** to add a new group of conditions and inner groups. The relationship between two or more outer groups is always OR.

Keep Track of Per-Tenant Alert Filter Statistics

By default, the Alert Filters table displays a **Hit Count** column that helps you keep track of how many alerts have been excluded by an alert filter on a per-tenant basis. Use this feature as follows:

1. Select **System | Saved Objects | Queries and Filters | Alert Filters** .
2. Locate the **Hit Count** column.

   If it does not appear, you can add it from the column picker to the left of the table.
3. Select in the **Hit Count** column header to sort the table by the number of alert filters with the most hits.
4. After sorting the table so you can see entries with non-zero values in the **Hit Count** column, select any entry in the column to see the number of alerts excluded by the filter broken out by tenant.

Set an Expiration Period

You can configure an alert filter to remain active only during a specified period of time.

6.5.0 This is useful when you want an alert filter action to apply temporarily during a maintenance window, a planned change, an incident mitigation period, or another short-term operational need. For example, you can temporarily suppress expected alerts, apply different severity scoring during a known event, prevent selected alerts from affecting associated cases for a limited time, or add temporary tags that help with investigation and reporting. After the specified end time passes, Stellar Cyber deactivates the alert filter automatically, so you don't need to return later to remove the temporary configuration.

To add an expiration to an alert filter:

1. Create a new alert filter or edit an existing one.

2. Select **+ Add expiration**.

   A date picker appea

3. Select the date when you want the alert filter to expire.

   The alert filter will expire at 23:59:59 on the selected date.

   The time zone of the expiration date that the UI displays is learned from the time zone of your browser.

4. Enable **Notify on activation**, **Notify on deactivation**, or both depending on when you want to be notified with status updates.
5. **Save** the alert filter.

While the alert filter is active, Stellar Cyber applies its configured action to matching alerts. When the alert filter expires, Stellar Cyber deactivates it automatically and normal alert processing resumes.

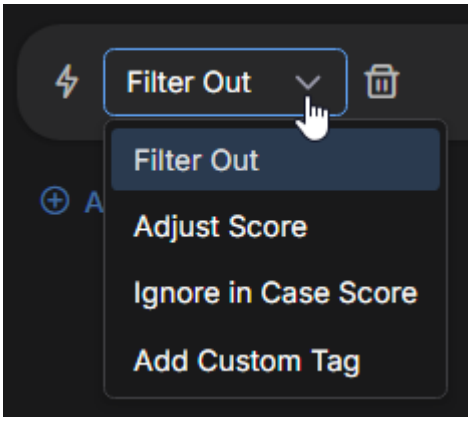
Actions 6.2.0

The alert filter builder supports multiple actions that let you suppress alerts, adjust alert severity scores, control whether alerts contribute to the severity score of the cases they are assigned to, and enrich alerts with custom tags. When Stellar Cyber detects one or more conditions that match an alert filter, it automatically applies the selected action or actions.

To see the available actions, select **+ Add Action**.



Then expand the **Add Action** drop-down list that appears.



Filter Out

The **Filter Out** action suppresses alert creation when alerts are not relevant. Use this action to reduce noise in environments where suspicious activity is expected or planned. Suppressed alerts do not appear on the **Alerts** page, which means you can't combine it with any of the other alert actions. **Filter Out** is the default action and is applied to all existing alert filters when you upgrade from a release earlier than 6.2.0.

Examples of when you might suppress the creation of alerts:

- **Lab networks**: Suppress alerts from test ranges or sandbox tenants where anomalous behavior is expected.
- **Security tools noise**: Suppress alerts from vulnerability scanners or red team exercises.
- **Tenant exercises**: Suppress alerts in MSSP tenant environments during planned simulations.

Adjust Score

The **Adjust Score** action raises or lowers the severity of alerts instead of suppressing them. You can raise a severity score by +10%, +20%, or +30%, or lower it by -10%, -20%, or -30%. This provides flexibility to align alert scoring with business priorities and the relative importance of different network entities. Adjustments are visible on the **Alerts** page.

When an alert filter uses the **Adjust Score** action to change the score of an alert, the score of any associated open case is also recalculated to reflect the updated alert score. 6.5.0 This ensures that case severity remains aligned with the current severity of the alerts it contains. New cases use the adjusted alert scores when they are created, and existing open cases update automatically when alert scores change.

This action cannot be combined with **Filter Out** or **Ignore in Case Score** because those actions contradict the purpose of adjusting severity. However, you can combine **Adjust Score** with **Add Custom Tag** to add business context to alerts that have modified severity.

Examples of when you might adjust the severity score of alerts:

- **High-value resources**: Increase severity (+30%) for login anomalies on mission-critical servers, domain controllers, or VIP accounts.
- **Low-value resources**: Decrease severity (–30%) for alerts on non-critical assets such as DHCP servers on guest networks.
- **Cloud assets**: Boost severity for anomalies tied to production workloads while reducing severity for alerts tied to staging or sandbox environments.



See "Add Custom Tag" (below) for examples that combine **Adjust Score** and **Add Custom Tag** actions.

Ignore in Case Score

The **Ignore in Case Score** action keeps alerts visible in Stellar Cyber but excludes them from influencing the severity score of any case to which they belong. Use this action when alerts are useful for visibility but should not drive escalation. Alerts excluded in this way remain on the Alerts page but do not affect case scoring on the **Cases** page.

The **Ignore in Case Score** action cannot be combined with any other alert actions.

Examples of when you might want to prevent alerts from contributing to the severity score of cases:

- **Background noise alerts**: Retain IDS/IPS detections for visibility (such as outdated browser exploit attempts) without letting them inflate case severity.

- **Known benign behavior**: Exclude recurring alerts from IT automation scripts or vulnerability scans.
- **Tenant-specific exceptions**: For MSSPs, exclude alerts that are acceptable in one customer's environment (such as legacy protocol use in manufacturing networks) while keeping them logged.

Add Custom Tag

The **Add Custom Tag** action enriches alerts with business-specific context, making triage, investigation, and reporting more efficient. After choosing **Add Custom Tag** from the **Add Action** drop-down list, either choose an existing tag from the field that appears to the right or enter the name of a new tag, such as vip, in this field. You can create and choose multiple tags.

Predefined and custom tags appear on the **Alerts** page, can be used to organize cases on the **Cases** page, enable pivoting during investigations in Threat Hunting, and allow grouping in dashboards and reports for internal teams and MSSP customers.

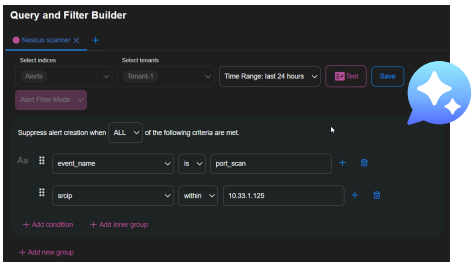
This action can be combined with **Adjust Score** but is not meaningful when combined with **Filter Out** or **Ignore in Case Score** so these combinations aren't allowed in the UI.

Examples of custom tags you might want to add to alerts:

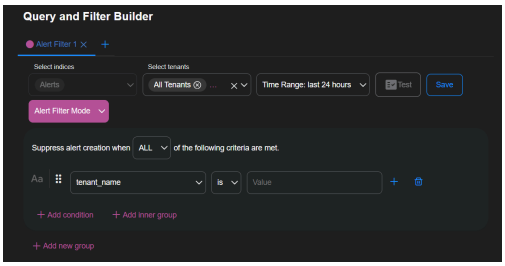
- **Business impact tags**: Tag ransomware detections with "Financial Loss," insider threat alerts with "Theft of Sensitive Information," and DDoS attempts with "Operational Disruption."
- **Business use case mapping (MSSP)**: Add tags such as "PCI Scope," "Customer Portal," or "Crown Jewel Asset" to help customers see which business area is affected.
- **Operational reporting**: Enable reporting that shows not only the number of alerts processed but also the threats tied to specific business risks.
- **Critical systems**: Use the **Adjust Score** action to increase severity by +30% for detections involving hostnames or IP addresses in a watchlist of mission-critical servers, and use the **Add Custom Tag** action to tag these alerts with "High-Value Asset." This ensures alerts for business-critical systems are emphasized during investigations and visible in filtered dashboards.
- **Safe environments**: Use the **Adjust Score** action to decrease severity by -20% for detections from known test networks or sandbox tenants listed in a watchlist, and use the **Add Custom Tag** action to tag these alerts with "Test Environment." This maintains visibility of expected activity while preventing these alerts from inflating operational metrics or case severity scoring.

Alert Filtering Examples

The following example filters out a vulnerability scanner from triggering port scan alerts.



The following example filters specific tenants. After selecting **All Tenants**, you can add a condition, search for the `tenant_name` field, then enter one or more values for specific tenant names.



Alert Filter Editability

Stellar Cyber enforces restrictions to ensure that partners and tenant users retain control over their alert filter configurations, even if a user with a higher scope creates similar filters. If an alert filter is created for a single tenant, it's not possible later to tell whether a root user, partner, or tenant user created the filter because they all have the ability to do so. After such a filter is created, Stellar Cyber makes the **Select tenants** field non-editable for root users and partners. This is to ensure that users with a higher-level scope don't transfer alert filters created by users at a lower level for their tenant to another tenant.

Similarly, if an alert filter is created for a single tenant group or multiple individual tenants, it's later not possible to tell if a root user or partner created the filter as both user types are capable of it. In this case, the **Select tenants** field becomes non-editable for root users but remains editable for partners, assuming the tenants fall within their assigned tenant group.

Root users are not restricted in editing the **Select tenants** field in an alert filter when it's for "All Tenants" or multiple tenant groups. Because they're the only ones whose scope is broad enough to have made these selections, it's clear that a root user can later edit the tenant selections without infringing on the configuration of a lower-scope user.

Using Case Filters and the Filter Builder

Case filters suppress the creation of cases based on criteria you define, enabling you to reduce case volume and focus attention on more relevant incidents. When alerts are grouped into a proto-case, Stellar Cyber first checks case filters before promoting it to a full case. If any filter matches, the system suppresses the case creation. You can create and manage case filters in the Queries and Filters Manager under **System | Saved Objects | Queries and Filters**, or directly from **Cases |** ⚙ (**Case Settings** icon) **| Case Filters | + Create** or **Edit**.

Case filters apply only to system-created cases. Stellar Cyber doesn't filter cases created manually by users because they are already visible to them.

- **Use the Case Filter Builder in the Query and Filter Manager**

- Use the Filter Builder in Case Settings
- Add Conditions and Groups
    - Case Filter Parameters: Condition Type
    - Add a Condition
    - Add an Inner Group and a New Group
    - Add a Sub-filter
- Case Filtering Examples

Use the Case Filter Builder in the Query and Filter Manager

The Case Filter Builder interface is consistent with other filter types in the platform, supporting structured condition-building, logic groups, and aggregations.

To create a case filter in the Query and Filter Manager:

1. Navigate to **System | Saved Objects | Queries and Filters**, select the **Case Filters** tab, and then select **+ Create** to open the Filter Builder in Case Filter Mode.

   In Case Filter mode, **Cases** is preselected as the index that's used as the data source and cannot be changed.

2. If you are logged in to Stellar Cyber at the root level, choose an individual tenant or **All Tenants**. This sets the scope of its availability.

   About Share Scope

   You can configure objects such as case filters to suppress the creation of cases. Root-level users can set the scope of these case filters as **All Tenants** or an individual tenant. If a root user creates a case filter for **All Tenants**, then it's available for every tenant to see and use, and if a root user creates a case filter for a specific tenant, then it's available for just the individually specified tenant. Partner-level users can create case filters whose scope is an individual tenant in their group. Tenant-level users can create case filters whose scope is their own tenant.

   Users at lower scopes can see and use case filters that apply to their tenants when these filters were created by users at higher levels. Partner- and tenant-level users see case filters that root-level users created for **All Tenants** and for individual tenants if the filters apply to them. Tenant-level users see case filters that partners created for their tenant.

   When partner- and tenant-level users use case filters that a root-level user created for **All Tenants**, they only filter cases that are within the sphere of their own tenant group (for partners) or tenant (for tenant users); they don’t filter cases that belong to any other tenant group or tenant. From the perspective of partners and tenant users, all case filters pertain only to cases within their own tenant group or tenant, regardless of the scope used when creating them.

   In summary:

- Root users can create case filters with the scope of **All Tenants** or an individual tenant.
- Partners can create case filters with the scope of a tenant in their tenant group.
- Tenant users can create case filters whose scope is their own tenant.
- When partner- and tenant-level users use case filters with a scope of “All Tenants”, they only filter cases pertaining to their tenant group (for partners) or tenant (for tenant users).

3. Set the operator (**AND**, **OR**, or **NOT**) to determine the relationship between conditions in the group.
4. Select **Add condition**.

   For details about defining conditions and adding more conditions, inner groups, and new groups of conditions, see Add Conditions and Groups in this topic.
5. Select **Test** and set a **Time Range** to test your configuration before saving it.

   When you select **Test**, Stellar Cyber evaluates your case filter against the 50,000 most recent cases in the selected time range. It displays a list of cases that would have been suppressed if the filter had been in place at the time these cases were created. This lets you quickly assess whether the filter is targeting the types of cases you want to suppress. You can review the test results, adjust the filter logic if needed, and continue testing until the outcome aligns with your intent.
6. Select **Save | Save As**, give the case filter a meaningful name, add a description about it for future reference, and then **Save**.

   Stellar Cyber saves the case filter and displays it in the same tab with its new name.

   By default, newly created filters are immediately enabled, and Stellar Cyber begins using them to suppress the creation of new cases that meet the criteria defined by the filter. However, enabling a new case filter does not retroactively remove or close existing cases that match its conditions. This safeguard ensures that ongoing investigations are not disrupted by filters applied after the fact.
7. To add another case filter, select the **Plus** icon (**+**) to the right of the active tab and select **New Case Filter**.

   Stellar Cyber keeps the tab for the first case filter open, creates a new tab next to it, and switches to the new tab. You can leave both the first and second tabs open and switch back and forth between them as you work.

    You can have up to ten tabs open simultaneously.

Use the Filter Builder in Case Settings

The embedded filter builder allows you to stay within the Case interface while configuring filter criteria. All changes here are reflected in the Queries and Filters Manager.

To manage filters from the Case Management interface:

1. Navigate to from **Cases |** (**Case Settings** icon) **| Case Filters | + Create** to create a new case filter or the **Edit** icon in the row of a case filter that you want to modify.
2. Enter a name for the case filter, a description, and if you're a root-level user or partner select either **All Tenants** or a specific tenant.

   This sets the scope of its availability.

   The name can be up to 100 characters long including spaces.
3. Define when the filter will suppress case creation by choosing if **ALL**, **ANY**, or **NONE** of the following conditions must be met.
4. Set the operator (**AND**, **OR**, or **NOT**) to determine the relationship between conditions in the group.
5. Select **Add condition** to start defining your condition.

   For details on defining a condition and adding more conditions, inner groups, and new groups, see the next section, **Add Conditions and Groups**, and the other sections in this topic.
6. **Submit**.

   As soon as you save a case filter, Stellar Cyber immediately begins using it to filter new cases. However, it does not apply a filter to existing cases because they might already be under investigation. You don't need to apply case filters manually; upon their creation, Stellar Cyber applies them automatically.

Add Conditions and Groups

Case filters in Stellar Cyber are constructed using Lucene-based logic within a graphical builder that supports a structured hierarchy of conditions and groups. These constructs allow you to define precise logic for suppressing the creation of unwanted cases. Within a case filter, you can define conditions, nest them in inner groups or new groups, and use sub-filters for specific field types. Each type of grouping has a distinct function in building complex logic.

A group consists of one or more conditions and can also contain inner groups and sub-filters to build complex logical expressions. Inner groups are dependent on their parent conditions whereas sub-filters are more like secondary standalone conditions. When there are multiple outer groups, each one operates independently and is connected by an "OR" relationship, meaning that if the conditions defined in any outer group are true, the overall case filter condition is satisfied. Within each group, the logical relationships among individual conditions and inner groups can be defined using "AND," "OR," or "NOT" to specify how the conditions should interact. Conditions within a group using "AND" must all be true for the group to match, "OR" allows any condition within the group to trigger a match, while "NOT" excludes any condition within a group from triggering a match. By combining these elements, you can create case filters that suppress the creation of cases matching your criteria so you can focus on more relevant security events.

When building a case filter, you can add conditions, inner groups, new groups, and sub-filters. There is no maximum number of components that you add to a case filter. However, the

maximum number of layers of nested inner groups is ten (not counting the outer group that encompasses the nest of ten inner groups within it).

Case Filter Parameters: Condition Type

The options are **AND**, **NOT**, and **OR**. These boolean operators are applied when the new condition is entered and define how conditions relate to one another when the filter is evaluated.

- **AND** returns only cases that meet all of the specified conditions.
- **NOT** excludes cases that meet the listed condition or group of conditions.
- **OR** returns cases that meet any of the specified conditions.

At the top (group) level, the operator determines how the entire set of subconditions is combined, while within a nested group, it defines the logical relationship among the individual subconditions inside that group.

The condition type does not apply to the first condition in a new group. The first condition defines the starting point of the group, so a logical operator is needed only when you add a second condition or create an inner group.

In the Stellar Cyber interface, the selector wording differs slightly from the logical terms stored in the underlying case filter condition markup:

Value in the UI	Value in the Condition Markup
NONE	NOT
ALL	AND
ANY	OR

For example, if a case filter is configured as—

> Find documents where **NONE** of the following criteria are met
>
> Field = assignee, Operator = is, Value = schmetterling

—the condition markup generated for this filter is:

```
{"conditions":
[{"caseSensitive":false,"field":"assignee","op":"==","type":"stri
ng","value":["schmetterling"]}],"modifier":"not"}
```

The difference is that the UI uses **NONE**, while the markup stores the logical equivalent **not**. This difference appears only in the top-level condition group; nested groups always use the standard logical operators (**AND**, **OR**, and **NOT**) directly.

Add a Condition

A condition is a specific criterion or set of criteria that data must meet to be included in the result set.

To add a condition to a group, select **Add condition** and then define its components: field, operator, and value (in some cases multiple values). You can add as many conditions as you like. The operator defines the relationship between the field and the value.



Use the operator `is` or `is not` only for exact full matches. Use `contains` or `does not contain` when the match can occur anywhere in a field.

Add an Inner Group and a New Group

An inner group is a logical container nested within a condition group that shares and depends on the parent condition's context. It enables the creation of compound conditions that refine logic without branching off into separate filter paths.

For example, an inner group might be used to further qualify alerts associated with a proto-case using nested logic like this:

- Suppress case creation when ALL of the following are true:
    - Condition A (for example, `max(alert.event_score) < 30`)
    - Inner group:
        - Condition B1
        - Condition B2

To add an inner group to a group, select **Add inner group**. Once you do, the original group becomes the outer group. You can add up (up to 10 inner groups). This lets you nest conditions, which allows for more nuanced filtering by combining different criteria.

Use the following group operators to define the logical relationship between conditions within a group—the conditions within an outer group and the conditions within an inner group:

- AND – retrieves results that satisfy all the conditions
- OR – retrieves results that satisfy at least one of the conditions
- NOT – excludes results that satisfy at least one of the conditions

A new group, on the other hand, defines a parallel grouping of conditions at the same level as the current group. It is logically distinct and does not inherit or depend on the structure of the previous group. Use a new group when branching your logic into separate paths that independently affect the case suppression decision. To add a new group of conditions and inner groups, select **Add new group**. The relationship between two or more outer groups is always OR.

Add a Sub-filter

A sub-filter is a special construct required when certain fields—such as alerts—are selected. In these cases, the condition must include an aggregation function—f(x)—with COUNT (default), MAX, MIN, SUM, AVG, and COUNT DISTINCT and be accompanied by a sub-filter, which defines additional filtering criteria for the dataset being aggregated.

Sub-filters are logically independent of the parent condition, functioning like secondary standalone filters that limit the scope of the aggregation. They do not depend on the parent group's logic but instead operate on the dataset before the aggregation is evaluated.

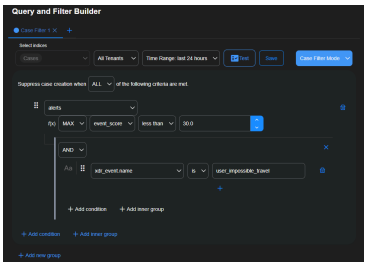
For example:

- Field: `alerts`
- Aggregation: `MAX (alert.event_score)`
- Sub-filter (required): Filter only alerts where `event_type is "impossible_travel"`

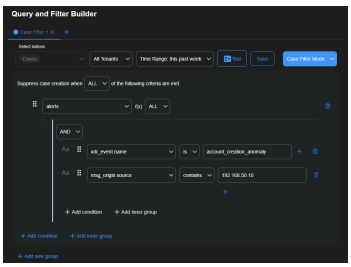
In this scenario, the sub-filter ensures that only alerts of a specific type are considered when calculating the maximum score used to evaluate case creation.

Case Filtering Examples

The following example shows a case filter designed to suppress case creation when only low-severity `user_impossible_travel` alerts are present. Specifically, the filter checks all alerts associated with a proto-case and calculates the maximum `event_score`. If that score is less than 30.0, and the alert type (`xdr_event.name`) is `user_impossible_travel`, the case is not created. This configuration uses an aggregation function (`MAX`) to evaluate alert severity and a sub-filter to limit the evaluation to a specific alert type—ensuring that only trivial instances of this behavior are filtered out, while higher-severity or different alert types still trigger case creation.



The following example shows a case filter that suppresses the creation of cases during scheduled account migration activities. It evaluates each alert in a proto-case using the `ALL` aggregation function and suppresses the case if all alerts meet two conditions: the `xdr_event.name` is `account_creation_anomaly`, and the `msg_origin.source` contains `192.168.50.10`—the IP address of a known Windows NXLog server responsible for forwarding account provisioning logs. This filter is typically disabled and only enabled during planned identity transitions, such as when acquired users are being migrated from one domain to another. Activating the filter during these periods prevents unnecessary case creation from predictable, high-volume provisioning activity that would otherwise trigger anomaly alerts.



© 2026 Stellar Cyber . All rights reserved.